



Asignatura: Programación II

Grupo: A, B, N

Docente: Ing. David Esteban Rodríguez Gómez

Estudiante: Juan Sebastian Quinto Hinestroza

Código: 1000338205

Taller Docker – Kubernetes Parte 1

Objetivo: Comprender, analizar y reflexionar sobre los principios, componentes y estrategias fundamentales que permiten garantizar la resiliencia, el balanceo de carga y el monitoreo eficiente en sistemas distribuidos de tipo escritorio.

1. Define qué es un sistema distribuido y explica por qué se vuelve más complejo gestionar la resiliencia en estos entornos comparado con sistemas centralizados.

Sistema distribuido: Conjunto de nodos independientes (servidores, microservicios, etc.) que colaboran para lograr un objetivo común, comunicándose a través de red.

Complejidad en resiliencia vs. sistemas centralizados:

- **Puntos de fallo múltiples:** Un fallo en cualquier nodo o en la red afecta al sistema completo.
- **Consistencia de datos:** Mantener datos sincronizados entre nodos es complejo (ej: particiones de red).
- **Coordinación:** Requiere protocolos adicionales (ej: elección de líder, consenso).

Ejemplo: En una base de datos centralizada, un fallo afecta todo el sistema; en una distribuida, un nodo puede caer, pero el sistema sigue funcionando (con posibles inconsistencias).

2. Explica con tus palabras qué es el balanceo de carga en aplicaciones de escritorio cliente-servidor y qué tipos existen.

Distribuir peticiones entre múltiples servidores para optimizar rendimiento y evitar sobrecarga.

Tipos:

- **Round Robin:** Asigna peticiones secuencialmente (ej: servidor A, B, C, A...).
- **Por peso (Weighted):** Prioriza servidores con más capacidad.
- **Por afinidad (Sticky Sessions):** Cliente específico siempre va al mismo servidor.

Ejemplo en escritorio: Una aplicación de atención al cliente que conecta a múltiples servidores locales para distribuir consultas.

3. ¿Qué diferencias existen entre alta disponibilidad y tolerancia a fallos? Da un ejemplo concreto para cada uno en un sistema de base de datos distribuida.



Asignatura: Programación II

Grupo: A, B, N

Docente: Ing. David Esteban Rodríguez Gómez

Estudiante: Juan Sebastian Quinto Hinestroza

Código: 1000338205

- **Alta disponibilidad:** Minimiza tiempo de inactividad (ej: réplicas de PostgreSQL que toman el control si el primario falla).
- **Tolerancia a fallos:** Continúa operando a pesar de fallos (ej: Cassandra sigue aceptando escrituras incluso con nodos caídos, usando consistencia eventual).

Ejemplo concreto:

- **HA:** Un clúster de MongoDB con réplicas que elige un nuevo primario automáticamente.
- **Tolerancia a fallos:** DynamoDB aceptando operaciones durante una partición de red, aunque algunos datos queden inconsistentes temporalmente.

4. Describe cómo funcionaría una política de failover en un clúster local con tres nodos que ejecutan servicios replicados.

Funcionamiento:

1. **Monitorización:** Un servicio (ej: Pacemaker) verifica latidos (heartbeats) entre nodos.
2. **Detección de fallo:** Si el nodo primario no responde en X segundos, se declara caído.
3. **Elección de nuevo líder:** Usando consenso (ej: algoritmo Raft) o prioridad configurada.
4. **Reinicio de servicios:** Los servicios del nodo caído se migran automáticamente a otro nodo.

Ejemplo: Un clúster de PostgreSQL con repmgr para failover automático.

5. ¿Qué papel cumple la replicación activa-activa en la recuperación ante fallos?

Papel: Permite que todos los nodos acepten operaciones (lecturas/escrituras), mejorando:

- **Disponibilidad:** No hay interrupción si un nodo falla.
- **Rendimiento:** Carga distribuida.
- **Recuperación:** Los nodos sincronizan datos cuando se reintegran.

Ejemplo: Bases de datos como Cassandra o Couchbase, donde cualquier nodo puede atender peticiones.

6. Analiza un caso donde la ausencia de monitoreo en un sistema distribuido provocó una falla mayor. Describe cómo se habría evitado.



Asignatura: Programación II

Grupo: A, B, N

Docente: Ing. David Esteban Rodríguez Gómez

Estudiante: Juan Sebastian Quinto Hinestroza

Código: 1000338205

Caso real: Caída de AWS US-East-1 (2021) por saturación de API, agravada por falta de alertas tempranas.

Cómo evitarlo:

- **Monitoreo proactivo:** Alertas en métricas como latencia API o tasa de errores.
 - **Circuit breakers:** Para evitar cascadas de fallos.
 - **Pruebas de carga:** Simular tráfico alto anticipadamente.
7. Define qué son los logs estructurados y cómo ayudan a la observabilidad en aplicaciones distribuidas de escritorio.

Definición: Logs en formato estándar (ej: JSON) con campos fijos (timestamp, level, user, etc.).

Beneficios:

- **Búsqueda eficiente:** Filtrado por campos específicos.
- **Correlación:** Agrupar logs de múltiples nodos (ej: con trace_id).
- **Integración:** Compatible con herramientas como ELK o Loki.

```
{
  "timestamp": "2023-10-05T12:00:00Z",
  "level": "ERROR",
  "user": "admin",
  "event": "login_failed",
  "ip": "192.168.1.1"
}
```

8. ¿Qué ventajas tiene usar herramientas como Prometheus y Grafana en lugar de monitoreo manual mediante logs locales?

Ventajas:

- **Visualización centralizada:** Gráficos y dashboards unificados.
 - **Alertas automáticas:** Basadas en umbrales (ej: CPU > 90%).
 - **Histórico:** Retención y análisis de tendencias.
- Ejemplo:** Detectar picos de RAM en tiempo real vs. revisar logs manualmente en cada nodo.

9. Compara los conceptos de reintentos automáticos y circuit breakers en entornos distribuidos. ¿Cuál aplicarías si un servicio de escritorio tarda en responder?

- **Reintentos:** Reintentar una operación fallida (ej: HTTP 503). Útil para fallos transitorios.



Asignatura: Programación II

Grupo: A, B, N

Docente: Ing. David Esteban Rodríguez Gómez

Estudiante: Juan Sebastian Quinto Hinestroza

Código: 1000338205

- **Circuit breaker:** "Abre el circuito" tras múltiples fallos, evitando saturación.
¿Cuál usar?: Para un servicio lento, **circuit breaker** (evita colapsar el sistema con reintentos infinitos).

Ejemplo:

```
// Circuit breaker con opossum (Node.js)
const breaker = new CircuitBreaker(asyncFunc, {
  timeout: 3000,
  errorThresholdPercentage: 50,
  resetTimeout: 30000
});
```

10. Explica cómo aplicarías un esquema de canary deployment para un sistema de atención al cliente instalado en múltiples nodos locales.

Implementación:

1. **Versión estable:** Todos los nodos ejecutan v1.0.
2. **Despliegue progresivo:**
 - 5% de nodos reciben v2.0 (canary).
 - Monitorizar métricas (errores, rendimiento).
 - Si todo OK, escalar al 100%.

Herramientas: Kubernetes (con canary deployments) o scripts personalizados.

11. ¿Por qué es importante aplicar políticas de afinidad en la distribución de procesos en entornos físicos multi-nodo?

Importancia: Optimiza uso de recursos al asignar procesos relacionados a nodos cercanos (ej: misma zona de red).

Ejemplo:

- **Afinidad por datos:** Procesos que usan la misma BD se ejecutan en nodos con acceso local a ella.
- **Evita latencia:** Comunicación entre servicios frecuentemente acoplados.

12. Expón cómo implementarías un plan de recuperación ante desastres para una aplicación de escritorio crítica conectada por red local.

Pasos:

1. **Backups diarios:** De BD y configuración (ej: con pg_dump o Veeam).



Asignatura: Programación II

Grupo: A, B, N

Docente: Ing. David Esteban Rodríguez Gómez

Estudiante: Juan Sebastian Quinto Hinestroza

Código: 1000338205

2. **Réplica en sitio secundario:** Espejo de servidores en otra ubicación física.
3. **Automatización:** Scripts para restaurar servicios rápidamente (ej: Ansible).
4. **Pruebas periódicas:** Simular fallos y validar recuperación.
Ejemplo: Una app de hospital que prioriza restaurar el módulo de historiales médicos en <1 hora.

13. ¿Cómo se interpreta un perfil de rendimiento (CPU, memoria) en aplicaciones distribuidas? ¿Qué decisiones operativas pueden derivarse de él?

Interpretación:

- **CPU alto en un nodo:** Posible cuello de botella (escalar horizontalmente).
- **Memoria creciente:** Fuga de memoria (investigar servicio específico).

Decisiones:

- **Escalar:** Añadir nodos si la carga es uniforme.
- **Optimizar:** Rebalancear servicios si un nodo está sobrecargado.

14. Describe las consecuencias de una mala configuración de timeouts en la comunicación entre procesos distribuidos.

- **Timeout muy bajo:** Falsos positivos (se cancela una operación que podría haber terminado).
- **Timeout muy alto:** Bloqueo prolongado de recursos (ej: hilos esperando respuesta).
Solución: Ajustar según percentiles históricos (ej: P99 de latencia + margen).

15. Explica el concepto de fallback y diseña un escenario realista en donde su uso sea más ventajoso que detener el proceso ante errores.

Definición: Mecanismo alternativo cuando un servicio falla (ej: cache local si la BD no responde).

Escenario realista:

- **Contexto:** App de ventas que depende de un servicio de tarjetas de crédito externo.
- **Fallback:** Usar un método de pago alternativo (ej: PayPal) o guardar la transacción localmente para procesarla luego.
- **Ventaja:** Mejor experiencia de usuario vs. mostrar error "Sistema no disponible".

Implementación:



Asignatura: Programación II

Grupo: A, B, N

Docente: Ing. David Esteban Rodríguez Gómez

Estudiante: Juan Sebastian Quinto Hinestroza

Código: 1000338205

```
// Ejemplo en Java con Resilience4j
CircuitBreaker.withFallback(
    () -> servicioExterno.comprar(),
    () -> guardarEnColaLocal() // Fallback
);
```